Phantom Events: Demystifying the Issues of Log Forgery in Blockchain

Yixuan Liu, Yuxin Dong, Ye Liu, Xiapu Luo, *Senior Member, IEEE,* and Yi Li, *Member, IEEE*

***Abstract*—With the rapid development of blockchain technology, transaction logs play a central role in various applications, including decentralized exchanges, wallets, cross-chain bridges, and other third-party services. However, these logs, particularly those based on smart contract events, are highly susceptible to manipulation and forgery, creating substantial security risks across the ecosystem. To address this issue, we present the first in-depth security analysis of transaction log forgery in EVM-based blockchains, a phenomenon we term Phantom Events. We systematically model five types of attacks and propose a tool designed to detect event forgery vulnerabilities in smart contracts. Our evaluation demonstrates that our approach outperforms existing tools in identifying potential phantom events. Furthermore, we have successfully identified real-world instances for all five types of attacks across multiple decentralized applications. Finally, we call on community developers to take proactive steps to address these critical security vulnerabilities.**

I. Introduction

Blockchain is a decentralized digital ledger technology that securely records and verifies transactions across multiple computers, ensuring data integrity and transparency [47]. Bitcoin (BTC), the most prominent token in the blockchain ecosystem, reached a remarkable market capitalization of $1,800 billion in November 2024, demonstrating the impact and success of this technology [48].

Building on blockchain technology, Decentralized Applications (DApps) have emerged as a key innovation [42]. These applications run on smart contracts deployed across various blockchain networks, such as Ethereum, Polygon, and Binance Smart Chain (BSC). The expansion of DApps reflects a shift toward decentralized application development, providing diverse services such as Decentralized Finance (DeFi) and Gamefi. This evolving ecosystem is supported by a wide infrastructure, including cryptocurrency wallets and blockchain explorers, which contribute to the dynamism of the blockchain landscape [43].

Smart contract events and transaction logs are essential for tracking actions within blockchain applications, supporting functionalities in cross-chain bridges, wallets, and NFT marketplaces. Falsifying these logs can undermine security, leading to asset theft, fraud, and interoperability issues that threaten user trust. While past research has examined vulnerabilities in specific domains, such as cross-chain bridges [24], [53], [54], NFT ownership [46], code inconsistencies [5], and token behaviors [59], a comprehensive study on event forgery across multiple applications remains absent. This paper addresses this gap by introducing *Phantom Events*, a new class of vulnerabilities that reveal the widespread risks of *log forgery* on the blockchain. Unlike previous studies on isolated scenarios, our work systematically summarizes event misuse across decentralized applications and explores distinct detection methods to identify phantom events. In prior studies, detection tools such as XScope [24] and XGuard [54] operate at the source code level, limiting detection when only bytecode is available. Conversely, SmartAxe [53] and TokenScope [59] analyze bytecode but their bytecode-only approach misses event parameter details. Guidi et al. [46], which focuses solely on transaction logs, experiences high false-positive rates due to limited contextual analysis. Our tool combines bytecode, source code (when available), and transaction data, enabling comprehensive detection across multiple vulnerabilities while reducing the false positives that affect single-layer analysis methods.

Phantom events are subtle manipulations within blockchain systems that turn normal contract events into hidden channels for unauthorized actions. These events compromise trust in blockchain transactions, enabling unauthorized activities that bypass typical security checks. They either mimic or slightly alter smart contract events to mislead event listeners, user interfaces, and blockchain analysis tools. This exploitation damages data integrity and can lead to substantial financial and reputational losses. For example, on 9 February 2024, attackers caused a 1.04 million USD loss by forging transfer events [49].

Our research focuses on understanding and categorizing the various types of phantom events. We conducted a comprehensive survey of recent blockchain security reports and articles to gather real-world attacks that exploit vulnerabilities related to phantom events. Furthermore, we audited multiple active smart contracts and developed a security model to process blockchain events (Sect. III), which serves as the foundation for analyzing and categorizing the collected attack scenarios (Sect. IV). Furthermore, we developed a tool named PEventCatcher, designed to detect vulnerabilities related to phantom events (Sect. V). Designing PEventCatcher presented several challenges: (1) many contracts exist only as bytecode, lacking event semantics, which complicates analysis; (2) phantom events mimic the format of legitimate events, making them hard to discern; and (3) understanding the business logic of contracts is necessary to determine the presence of irregularities.

Using our security model and detection tool, we identified nu-

Yixuan Liu and Yi Li are with College of Computing and Data Science, Nanyang Technological University, Singapore.

Yuxin Dong is with School of Software and Microelectronics, Peking University, Beijing, China.

Ye Liu is with School of Computing and Information Systems, Singapore Management University, Singapore.

Xiapu Luo is with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong.

merous vulnerabilities and potential risks in various blockchain applications (details reported in Sect. VI). Our results reveal that phantom event vulnerabilities are widespread and affect a variety of applications in the blockchain ecosystem. Through our auditing efforts, we discovered previously unreported instances of inconsistent logging vulnerabilities, where on-chain issues resulted in discrepancies with off-chain records. Furthermore, our tool identified historical event issues on chain that had gone undetected, underscoring the effectiveness of the tool in uncovering past incidents. In contract-level analysis, our tool outperformed existing solutions, demonstrating superior accuracy in detecting phantom event vulnerabilities. Furthermore, we identified security issues in multiple real-world applications, including cryptocurrency wallets, blockchain explorers, and cross-chain bridges, and reported these findings to project teams, claiming bounties for responsible disclosure. Finally, we propose mitigation strategies to address these vulnerabilities in Sect. VIII.

In short, we make the following contributions in this paper:

- **Novel Attack Taxonomy.** We are among the first to systematically analyze and categorize attacks related to the forgery of blockchain transaction logs. We propose a security model that captures five types of attack. We also identified previously unreported inconsistent logging vulnerabilities, where on-chain issues result in discrepancies with off-chain records.
- **Efficient Detection.** We developed a detection mechanism that surpasses existing tools in identifying Phantom Events, successfully uncovering previously unreported attack transactions.
- **Practical Implications.** We identified real-world issues across various blockchain applications, including blockchain explorers, cryptocurrency wallets, GameFi, DeFi, and NFT marketplaces. Notably, we found a vulnerability in a cross-chain relayer, which is also used by other projects, including one with a market capitalization of over $250 million. To date, we have reported six cryptocurrency wallet issues, one blockchain explorer issue, and one cross-chain bridge issue to project teams and bug bounty platforms, with five confirmed and one resolved, earning a total of $600 bounty for responsible disclosure.

II. Background

A. Smart contract event and transaction log in EVM-based Blockchains

Smart contracts, written in high-level languages like Solidity [30] and Vyper [31], are compiled into bytecode for execution on the Ethereum Virtual Machine (EVM) [18]. EVM-based blockchains support a wide range of functionalities, from simple transfers to complex DApps [20]. In these blockchains, *events* and *transaction logs* enable communication between smart contracts and external systems. When a smart contract emits an event, it is recorded as a transaction log, involving key components as follows.

Event. An event, such as `Deposit`, is defined as a tuple $E_{\texttt{Deposit}}(P_1, P_2, \ldots, P_n)$, where the parameters P_i may be *indexed* (stored in *topics*) or *non-indexed* (stored in *data*). For example, in the event `Deposit(address indexed account, uint256 amount, uint256 timestamp);`, the `account` is an *indexed* parameter, which means it will be included in the event's topics, enabling more efficient searching and filtering in the event logs. The `amount` and `timestamp` parameters are not indexed, but they still provide valuable information about the transaction.

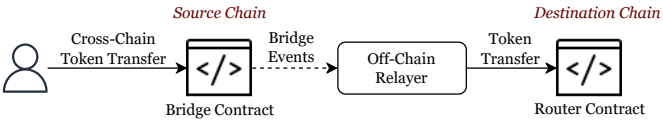


Fig. 1: The basic architecture of a cross-chain bridge.

Listener. An external system (e.g., a DApp or wallet) that monitors specific events by subscribing to logs through RPC methods.

Transaction. A transaction TX_{hash} represents an action on the blockchain, such as invoking a function $f_{\texttt{name}}$ within a smart contract. It includes the sender's and recipient's addresses, the transfer amount, and input data defining the function call and its parameters.

Sender. The sender TX_{sender} is the user address that creates the transaction.

Emitter. The emitter $S_{\texttt{address}}$ is the smart contract triggering the event upon meeting certain conditions.

Transaction Log. A log $L_{hash} = (Topics_0^i, Data^i, S^i_{\texttt{address}})$ is generated when an event is emitted during smart contract execution. $Topics_0^i$ represents the event signature for the i-th log, $Data^i$ contains non-indexed parameters, and $S^i_{\texttt{address}}$ is the contract emitting the event. These logs are stored on the blockchain and referenced for event monitoring.

B. Token

In EVM-based blockchains, tokens are implemented as smart contracts adhering to standards like *ERC-20* and *ERC-721*, which define interfaces and events to ensure compatibility across DApps. For instance, ERC-20 specifies events like `Transfer` and `Approval` for tracking token transfers and approvals, while *ERC-721* defines similar events but is adapted for NFTs, where each token is unique. These standardized events enable external systems, such as wallets and exchanges, to monitor token activities and update user balances or ownership records in real time, ensuring accurate and efficient tracking.

C. Cross-Chain Brdiges

Some DApps rely on off-chain modules to monitor transactions and events emitted by smart contracts in order to stay updated on on-chain activities and state changes in real time. For example, cross-chain bridges use these events to trigger off-chain processes, update user interfaces, or others.

In the case of a cross-chain bridge, as illustrated in Fig. 1, the smart contract on *source chain* acts as the emitter S_{source},

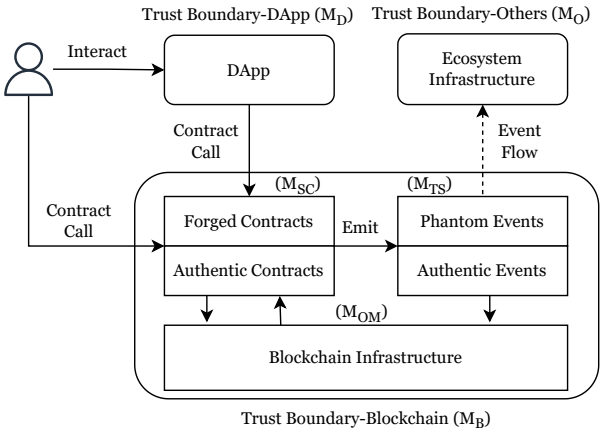


Fig. 2: Trust Boundaries in Blockchain Interactions

emitting specific events such as `Deposit` when users deposit assets. And the smart contract on the *destination chain* as an actor, being invoked by an off-chain relayer to process the deposited assets and complete the transfer operation.

When a user deposits tokens in a *bridge contract* in the source chain, the deposit event $E_{\texttt{Deposit}}$ is emitted and recorded as a transaction log $L_{deposit_{tx}}$. An *off-chain relayer*, functioning as a listener, monitors these logs to detect the `Deposit` event, process the event information, and coordinate with the smart contract on the destination chain to complete the asset transfer.

III. Threat Model and Motivating Examples

This section presents a threat model for the systematic analysis of event-related attacks and illustrates the impact of Phantom Events through motivating examples.

A. Trust Boundaries in Event Interactions

In the complex environment of blockchain interactions, establishing clear trust boundaries is essential for comprehensively understanding the security risks and vulnerabilities associated with transaction and event workflows. The interact model illustrated in Fig. 2 defines three primary trust boundaries:

Trust Boundary-Blockchain ($\mathcal{M}_B$). The core trust boundary is the blockchain, which serves as the backbone of the system where transactions and smart contract events are stored. Within this boundary lies the Transaction Storage ($\mathcal{M}_{TS}$), which can be divided into *phantom events* and *authentic events*. Phantom events are artificially created or manipulated events that may trigger unintended actions, while authentic events are the expected output of blockchain transactions. The Smart Contract area ($\mathcal{M}_{SC}$) can be divided into *authentic contracts* and *forged contracts*, indicating the potential for contracts to operate as intended or act maliciously. Other Blockchain Modules ($\mathcal{M}_{OM}$) include consensus mechanisms, node communication protocols, and other components that support blockchain functionality.

Trust Boundary-DApp ($\mathcal{M}_D$). The second boundary encompasses DApps that act as interfaces between users and the blockchain. DApps listen for events from the blockchain and respond accordingly, often triggering transactions or updates in response to these events.

Trust Boundary-Others ($\mathcal{M}_O$). The third boundary includes the broader ecosystem, such as off-chain services, external APIs, and wallets. These components also rely on the integrity of blockchain events for proper functionality.

B. Threat Model

In our threat model, attackers seek to compromise DApps and the broader ecosystem infrastructure by emitting forged events, and may also exploit these attacks to mislead users through social engineering tactics. While regular users emit legitimate events by invoking authentic contracts, attackers can emit forged events in several ways: by calling a forged contract, by calling an authentic contract, or by invoking a forged contract that subsequently calls an authentic contract to emit a forged event. The attacker's capabilities are confined to making direct contract calls or interacting with DApps.

C. Motivating Examples

Numerous real-world blockchain attacks have occurred due to the forgery of transaction logs. For example, the Qubit Bridge and pNetwork Bridge attacks [28], [44] caused $80 million and $4.3 million losses, respectively, due to the exploitation of Phantom Event vulnerabilities.

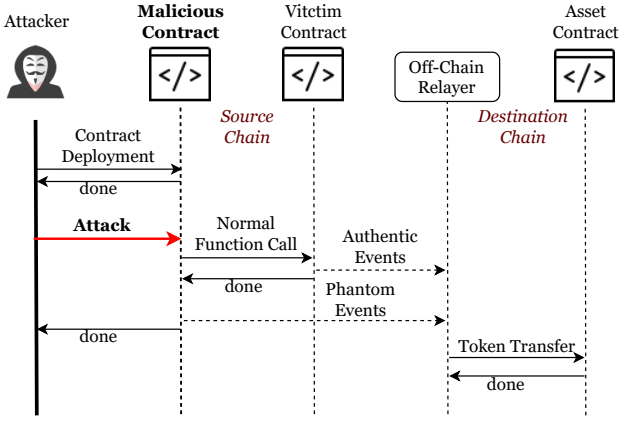


Fig. 3: Attack sequence of pNetwork Hack.

In the pNetwork hack, there is a flaw in the event handling mechanism, where if both a malicious contract and a legitimate contract are invoked within the same transaction and both emit events, the mechanism mistakenly treats all emitted events as legitimate contract events. So as shown in the attack sequence (Fig. 3), the attacker first deployed a malicious contract $S_{\texttt{malicious}}$ on the source chain which invoking the legitimate contract $S_{\texttt{legitimate}}$ through a function call. After the legitimate contract $S_{\texttt{legitimate}}$ executed its function $f_{\texttt{deposit}}$ and emitted the authentic event $E_{\texttt{Deposit}}$, the malicious contract $S_{\texttt{malicious}}$ simultaneously emitted a

counterfeit event $E_{\texttt{Deposit}}$ with an untruthful amount. Since both the authentic event and the counterfeit event were recorded under the same transaction $TX_{deposit}$, the relayer processed the counterfeit event as if it were legitimate. This allowed the attacker to transfer unauthorized funds to the destination chain by exploiting the inability of the system to differentiate between the two events.

In the Qubit Bridge attack, the attacker exploits a flaw in the `deposit` function of the victim contract, as illustrated in Fig. 4. The relayer transfers assets on the destination chain by retrieving the token address and amount from the event emitted on the source chain. However, the attacker exploited this by using the `deposit` function to emit an event that should only be emitted by the `depositETH` function, allowing them to trigger a transfer of ETH on the destination chain without actually depositing any ETH on the source chain, thus bypassing the relayer's checks and profiting from the attack.

Idea of PEventCatcher. Motivated by these real-world examples, we apply customized detection methods for various types of vulnerabilities. The phantom events may arise from issues in smart contract logic or from flaws in off-chain processors. To address this, we designed a smart contract-level approach, which identifies vulnerabilities that could lead to phantom events. Additionally, we developed an on-chain approach for transaction data, inspecting transactions for patterns or anomalies that may indicate attacks. These approaches complement each other and enable a comprehensive detection framework that addresses both contract-based and transaction-based origins of phantom events.

```
1  function depositETH(uint destinationChainId)
       external payable {
2    require(msg.value > 0, "Deposit amount must be
         greater than 0");
3    ethBalances[msg.sender] += msg.value;
4    emit Deposit(msg.sender, msg.value, address(0),
         destinationChainId);
5  }
6
7  function deposit(address token, uint amount, uint
       destinationChainId) external {
8    require(amount > 0, "Deposit amount must be
         greater than 0");
9    safeTransfer(token, address(this), amount);
10   tokenBalances[msg.sender][token] += amount;
11   emit Deposit(msg.sender, amount, token,
         destinationChainId);
12 }
13
14 function safeTransfer(address token, address to,
       uint value) internal {
15   (bool success, bytes memory data) = token.call(
         abi.encodeWithSelector(0xa9059cbb, to,
         value));
16   require(success && (data.length == 0 || abi.
         decode(data, (bool))), "!safeTransfer");
17 }
```

Fig. 4: Proof of concept for the event counterfeiting attack, where `deposit` and `depositETH` could emit the same event.

IV. Attack Taxonomy and Analysis

In this section, we present our taxonomy of attacks caused by phantom events and provide detailed explanations as well as detection rules for each attack.

We relied on industry reports, academic literature, and real-world smart contract audits to construct a taxonomy of relatively new and understudied phantom event attacks. We model five distinct attack scenarios, which are classified into two categories based on their origins: (1) *on-chain (smart contract) vulnerabilities* and (2) *off-chain weaknesses*. These attacks are summarized in our attack classification table (Table I).

Detailed demonstrations of these attacks are available in the supplementary materials.[1]

A. Attack Vectors and Detection Rules

This section introduces five attacks related to phantom events and proposes corresponding detection rules. Examples of these attacks can be found in Sect. A-A.

1) Event Counterfeiting: This attack takes advantage of legitimate contracts that emit the same event from multiple functions or execution paths ($\mathcal{M}_{TS}$). In the provided code example Fig. 4, both the `depositETH` and `deposit` functions emit a `Deposit` event, where the third parameter signifies the token type. While `depositETH` uses `address(0)` to represent ETH, the `deposit` function uses the token address for ERC-20 tokens. However, the `deposit` function can also emit an event with `address(0)`, mimicking an ETH deposit.

An attacker could exploit this by calling the `deposit` function and forging the third parameter as `address(0)`, without actually transferring any ETH. This creates a *phantom* `Deposit` event that resembles a real ETH deposit. Off-chain validation systems, which rely on event logs for transaction verification, may mistakenly treat this event as a legitimate ETH deposit, allowing the attacker to fraudulently claim large sums on the destination chain due to the flawed event validation process.

Detection Method. This vulnerability occurs when different functions or execution paths emit the same event with identical parameters, even though each function imposes different constraints. Off-chain systems often treat these events as authentic without verifying the parameter constraints. If a function emits a value that is expected to be constrained or validated by another function, it is considered a potential vulnerability. The detection rule checks for cases where identical parameter values are emitted along different execution paths, even when those paths should enforce distinct constraints, and flags such events as potential vulnerabilities.

The formalized detection rule is expressed as follows.

$$\text{EC} = \left\{ e \in E \mid \begin{array}{l} V(f,e) \cap V(g,e) \neq \emptyset \text{ and} \\ \exists\, f, g \in F(e) \end{array} \right\} \tag{1}$$

- E is the set of emitted events, and $e \in E$ represents an individual event.
- F is the set of all function paths in the contract that can emit events, and $f, g \in F$ are specific function paths that emit the same event e.

[1] https://github.com/PhantomEvent/Event-attack-demo

TABLE I: Mapping relationship between attack vectors and related modules.

Level	Attacks	Descriptions	Related Modules
On-chain	event counterfeiting	Use existing contracts to emit Phantom events	$\mathcal{M}_{TS}$, $\mathcal{M}_{O}$
	inconsistent logging	Inconsistencies between database and blockchain event emissions	$\mathcal{M}_{SC}$, $\mathcal{M}_{TS}$, $\mathcal{M}_{D}$
Off-chain	contract imitation	Deploy a malicious contract to emit Phantom events	$\mathcal{M}_{SC}$, $\mathcal{M}_{TS}$, $\mathcal{M}_{O}$
	transfer event spoofing	Emit Phantom events with social engineering attack	$\mathcal{M}_{SC}$, $\mathcal{M}_{TS}$, $\mathcal{M}_{O}$
	event handling error	Incorrect display/ insertion/storage due to faulty event processing	$\mathcal{M}_{D}$, $\mathcal{M}_{O}$

```
1 function requestWithdraw(uint256 _type, uint256
      _amount) external {
2   require(WITHDRAW_ALLOWED, "TroyEmpire:
        Withdrawal is disabled for now");
3   emit WithdrawalRequested(_msgSender(), _type,
        _amount);
4 }
```

Fig. 5: Proof of concept for the inconsistent logging attack.

- $V(f, e)$ and $V(g, e)$ represent the range of values that the parameters of event e can take along function paths f and g.

2) Inconsistent Logging: Many DApps adopt a hybrid logging model that uses both blockchain and traditional databases to record critical operations. For example, DeFi platforms record financial transactions, such as deposits and withdrawals, while GameFi platforms track user operations. Our analysis revealed that numerous projects not only rely on database records, but also emit logs on the blockchain.

Specifically, poorly designed smart contracts may allow attackers to emit forged events, leading to inconsistencies between the blockchain and the database logs. For example, a financial platform might use both blockchain and off-chain databases to log withdrawal requests. If the smart contract lacks proper access controls, attackers could arbitrarily generate withdrawal events on the blockchain, creating a mismatch with the database records, as shown in Fig. 5.

Detection Method. This vulnerability occurs when smart contracts emit events without proper access control or validation against stored data. Specifically, some contracts lack necessary constraints, allowing any user to trigger critical events such as withdrawals without validating the event parameters against the contract's state. To detect this, we analyze the contract's functions for two key aspects: (1) the presence of access control mechanisms or event parameter validation (e.g., `require` statements) that restrict who can emit events, and (2) whether the function interacts with storage (i.e., reads from or writes to storage variables) when emitting events to ensure that parameters are validated against previously stored values. If a function emits an event without adequate access control or does not interact with storage, it is flagged as a potential vulnerability.

The formalized detection rule is expressed as follows.

$$\text{IL} = \left\{ f \in F \mid \begin{array}{l} Constraint(f) = \emptyset \text{ or} \\ (S_{\texttt{read}}(f) = \emptyset \text{ and } S_{\texttt{write}}(f) = \emptyset) \end{array} \right\} \tag{2}$$

- $Constraint(f)$: The access control mechanism for function f or constraint for event parameters. If $Constraint(f) = \emptyset$, there are no access controls or constraints.
- $S_{\texttt{read}}(f)$ and $S_{\texttt{write}}(f)$: The storage variables read and written by the function f.

3) Contract Imitation: This attack involves the deployment of a malicious contract to exploit or imitate an existing contract ($\mathcal{M}_{SC}$). We classify this attack into two subtypes based on its characteristics. The first subtype, called Blended Event Attack, occurs when a malicious contract interacts with a legitimate contract, causing events from both contracts to be logged within the same transaction. This blending of events makes it difficult to distinguish between legitimate and fraudulent activity. The second subtype, named Mimicry Contract Attack, involves an attacker deploying a forged contract that imitates the behavior of a legitimate contract, allowing the attacker to manipulate event logs and transaction details.

An *authentic* contract typically records logs from its own functions. However, most contracts permit external calls, allowing malicious contracts to invoke their functions, leading to logs from both contracts being recorded in a single transaction ($\mathcal{M}_{TS}$). This scenario, termed the *Blended Event* attack, introduces a security risk when the log emitter is not verified.

The *Mimicry Contract* attack exploits transparency practices, as many DApp teams publish their contract source code to promote trust. This allows attackers to modify and redeploy code, creating malicious contracts that mimic legitimate ones and emit event logs that can be freely manipulated ($\mathcal{M}_{TS}$). For ERC-20 tokens or NFTs, this manipulation enables attackers to forge transaction records, such as minting or transferring tokens or NFTs, which can include falsifying the sender to make an NFT appear as though it was issued by a well-known artist.

Detection Method. To detect this, we analyze the *transaction logs*. Specifically, we check whether the same event signature ($Topics_0$) appears multiple times in the same transaction, but is emitted by different contracts. If the event signature is emitted by both the authentic contract and a forged contract within the same transaction, it is flagged as a potential attack transaction. This detection rule helps identify cases where the event signatures match but are emitted by different emitters, indicating a *Blended Event Attack*.

$$\text{BE} = \left\{ L_{hash} \mid \begin{array}{c} Topics_0^i = Topics_0^j \text{ and} \\ S^i_{\texttt{address}} = S_{\texttt{auth}} \text{ and} \\ S^j_{\texttt{address}} = S_{\texttt{forge}} \end{array} \right\} \tag{3}$$

4) Transfer Event Spoofing: This attack is a form of social engineering attack [21]. One common example is the Zero Transfer Scam, where the attacker creates a phantom event that mimics a legitimate token transfer. After the user sends tokens, the attacker generates a fake event that makes it appear as though the user has sent tokens to a similarly named recipient address, which belongs to the attacker. This counterfeit event is recorded by wallets and blockchain explorers, misleading the victim into transferring additional funds to the attacker's account. This scam has reportedly caused losses of at least $27.36 million USD, affecting 28,414 victims [51].

The second variation, called the Airdrop Scam [6], exploits the common Web3 marketing tactic of token airdrops. Attackers forge phantom events by spoofing the sender's address using real ERC-20 or ERC-721 contracts, often selecting addresses designed to catch attention (e.g., 0x8888...8888). These phantom airdrops trick victims into interacting with the tokens. Attackers may set up honeypot contracts where users can buy tokens but are unable to sell them, or use phantom event logs to promote malicious links (e.g., using ENS names to deceive victims). In more severe cases, attackers may perform rug pulls, withdrawing liquidity and leaving victims with worthless tokens [29].

Detection Method. Attackers exploit the fact that many third-party services, such as wallets and blockchain explorers, blindly trust events emitted by smart contracts without verifying their authenticity. By taking advantage of this lack of verification, attackers can generate phantom events that mimic legitimate transfers, misleading users into transferring funds to the attacker's address. To detect this, we can analyze the transaction logs to ensure that the transfer event was genuinely initiated by the correct sender, or verify if the sender had approved the address that initiated the transaction.

$$\text{TS} = \left\{ L_{hash} \mid \begin{array}{l} TX_{sender} \neq \textit{TokenSender} \text{ and} \\ Approve(\textit{TokenSender}, TX_{sender}) = \text{false} \end{array} \right\} \quad (4)$$

- *TokenSender* represents the sender of the tokens in event.
- $Approve(\textit{TokenSender}\ TX_{sender})$ indicates whether the token sender has authorized the transaction initiator to transfer their tokens.

5) Event Handling Error: This attack exploits vulnerabilities in how blockchain explorers, wallets, and monitoring tools process and interpret transaction data. These applications rely on on-chain data to provide real-time information to users. When phantom events are mistakenly treated as legitimate, this can result in inaccurate wallet balances, false transaction histories, or incorrect asset ownership records, which misleads users and potentially impacts their decision-making.

For example, in the context of *Contract Imitation* attacks, even if phantom events are not specifically targeting blockchain explorers, they may still cause explorers to log these events as valid ERC-20 or NFT transfers, despite no actual transfer occurring. This misinterpretation allows attackers to create the appearance of transfers without underlying transactions.

Attackers can further exploit this by embedding malicious payloads within event data to launch attacks such as cross-site scripting (XSS) or SQL injection (SQLi). Without proper sanitization, such payloads can lead to unauthorized actions on the DApp interface, data theft, or database compromise.

Detection Method. This attack leverages the implicit trust that blockchain explorers, wallets, and monitoring tools place in on-chain data. Effective detection requires adaptable methods tailored to the needs of each off-chain application. Explorers should validate events against actual token transfers, while wallets need to confirm that transfers are permitted for the sender. By customizing detection rules based on these unique validation standards, applications can mitigate the risks posed by phantom events and embedded malicious payloads.

V. Vulnerability Detection

In this section, we focus on detecting vulnerabilities related to the attack vectors discussed in Sect. IV. We introduce our hybrid approach, which combines static analysis with symbolic execution and on-chain data monitoring to ensure comprehensive vulnerability detection.

A. PEventCatcher *Framework Design*

As shown in Fig. 6, our detection approach employs a multi-level strategy, encompassing transaction-level, bytecode-level, and source code-level for vulnerability detection.

At the transaction level, we manually researched documentation for different projects to establish appropriate rules, given the variation in authentic events and emitters. These rules are applied to both historical and real-time on-chain transaction data to detect *Contract Imitation* by identifying instances where phantom events are blended with authentic ones.

For smart contracts, our analysis proceeds across bytecode and source code levels. Initially, we decompile smart contract bytecode into an intermediate representation (IR) in three-address form using Gigahorse [52]. From this IR, we construct an inter-contract control flow graph (ICFG) based on basic blocks. This graph enables backward taint analysis, which we use to trace event-related paths and detect vulnerabilities related to *Event Counterfeiting* and *Inconsistent Logging*.

If the source code is available, we extend our analysis using Slither to perform source code-level analysis. This provides a secondary confirmation for *Event Counterfeiting* by tracing event-related call paths and extracting constraints for symbolic execution, allowing us to verify parameter values more comprehensively.

Due to the unique nature of certain attacks, *Event Handling Error* lacks a specific detection method as it heavily relies on off-chain system validation. Similarly, *Transfer Event Spoofing* is primarily a social engineering attack that focuses on token behavior analysis. Therefore, our tool primarily targets detecting the remaining three types of attacks: *Event Counterfeiting*, *Inconsistent Logging*, and *Contract Imitation*, where distinct patterns and contract-level vulnerabilities can be effectively identified and addressed.

B. Transaction-Level Detection

At the transaction level, off-chain monitoring is applied to analyze on-chain transaction data and detect potential *Contract*

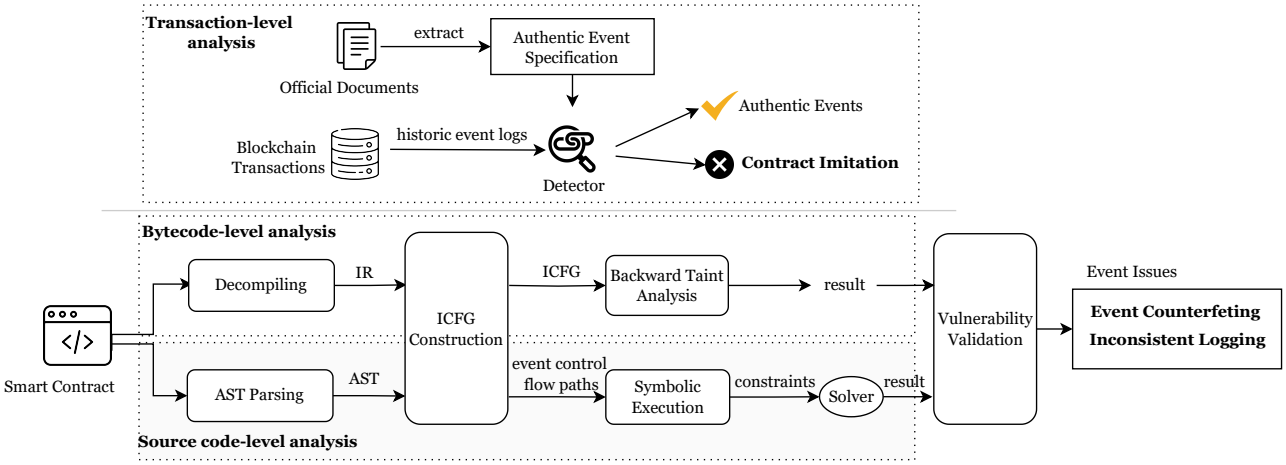


Fig. 6: The architecture of PEventCatcher.

Imitation attacks. By establishing domain-specific rules based on the expected behavior of smart contracts, we systematically verify whether transactions adhere to correct logic and identify any signs of malicious activity that may indicate phantom events or unauthorized interactions.

This analysis involves parsing transaction data and applying a set of rules to validate key aspects of events, including: (1) ensuring that the emitted event has the correct signature; (2) verifying that the event emitter matches the expected contract address; (3) checking that the function execution path aligns with the expected logic; and (4) confirming that the event parameter values match the expected ones.

By enforcing these rules, we can detect anomalies that may indicate *Contract Imitation* attacks. If an attack transaction is detected, further analysis is performed to confirm its impact, such as causing unauthorized fund transfers or state changes.

For example, in the pNetwork attack Table II, a *Redeem* event was emitted by a malicious contract with an unexpected contract address on the source chain. Applying these transaction-level rules allowed us to detect the attack. Subsequent monitoring of the destination chain transaction confirmed that the attack succeeded with unauthorized fund transfers.

C. Smart-Contract-Level Detection

At the smart-contract level, we detect vulnerabilities specifically related to *Event Counterfeiting* and *Inconsistent Logging* through a multi-tiered analysis approach consisting of two parts: (1) identifying potential phantom events through *interprocedural backward taint tracking* at the bytecode level, and (2) confirming vulnerabilities by *validating event parameter constraints* at the source-code level. This dual-method approach allows us to analyze different representations of the contract to identify vulnerabilities that may not be visible at a single level of abstraction.

1) Phantom Event Identification Through Interprocedural Backward Taint Tracking: At the bytecode level, as described in Alg. 1, the detection of vulnerabilities related to *Event Counterfeiting* and *Inconsistent Logging* is performed through backward taint analysis using an intermediate representation (IR) of the smart contract bytecode. The algorithm identifies vulnerable events E_v by analyzing the flow of data from event log operations to their sources and checking for specific conditions that indicate vulnerability.

The algorithm begins by initializing an empty set E_v to store vulnerable events (line 1). It constructs an inter-procedural

Algorithm 1 Detecting Inconsistent Logging and Event Counterfeiting via Backward Taint Analysis

Input: *IR*, an intermediate representation of smart contract bytecode via decompilation.
Output: E_v, a set of vulnerable events.

1: $E_v = \emptyset$.
2: **construct** *ICFG* out of IR.
3: **extract** *LogOps*, i.e., event log operations, from IR. ▷ Each operation contains a event signature and a set of logging data variables
4: *Vars* $= \emptyset$,
5: **for** logOp $\in$ *LogOps* **do**
6: Vars $\leftarrow$ Vars $\cup$ **vars**(logOp) ▷ Record all logging data variables
7: Slices $\leftarrow$ BackwardSlicing(logOp, ICFG) ▷ Each slice is a reverse execution flow path from logOp to function entry point
8: **extract** *Paths*, i.e., inter-functional paths based on Slices and ICFG
9: taintedPaths $\leftarrow \emptyset$
10: **for** $p \in$ Paths **do**
11: $source_{op} \leftarrow$ TaintAnalysis(p, Vars)
12: **if** $source_{op} \neq null$ **then**
13: **if sstore**($p[:source_{op}]$) $= \emptyset$ **then**
14: $E_v \leftarrow E_v \cup$ **event**($logOp$) ▷ No storage operations
15: taintedPaths $\leftarrow taintedPaths \cup \{$**event**$(logOp)\}$
16: **else**
17: **if** HasExternalCall(p) $\wedge$ not HasJumpi(p) **then**
18: $E_v \leftarrow E_v \cup$ **event**($logOp$) ▷ No constraints
19: **if** $||taintedPaths|| > 1$ **then**
20: $E_v \leftarrow E_v \cup$ **event**($logOp$) ▷ A event logging action has multiple tainted paths
21: **return** E_v
22: **procedure** TaintAnalysis(path, taintVars)
23: $entry = path[-1]$ ▷ Function entry point is the last item of this reverse execution path
24: **for** $instr \in path$ **do**
25: **if vars**($instr$) $\cap\, taintVars \neq \emptyset$ **then**
26: taintVars $\leftarrow$ taintVars $\cup$ **vars**($instr$) ▷ Taint propagation
27: **if** $vars(entry) \cap taintVars \neq \emptyset$ **then**
28: **return** entry
29: **else**
30: **return** null

TABLE II: Transaction logs of the pNetwork attack.

Events	Parameters	Emitters	Parameter Values
Burned	operator, from, amount, data, operatorData	pBTC	(Attacker, Attacker, 0, , _)
Transfer	from, to, amount	pBTC	(Attacker, address(0), 0)
Redeem	redeemer, value, underlyingAssetRecipient, userData	pBTC	(Attacker, 0, Attacker_Bitcoin_address, _)
Redeem	redeemer, value, underlyingAssetRecipient, userData	Malicious Contract	(Attacker, 274[...]144, Attacker_Bitcoin_address, _)

control flow graph (ICFG) from the IR (line 2) and extracts all event log operations (*LogOps*) from the IR (line 3). Each log operation consists of an event signature and a set of event data variables. The algorithm then initializes an empty set *Vars* to record the event variables (line 4).

The construction of the ICFG involves recovering function-level information, such as function names and event signatures, by matching hashed identifiers in the bytecode with entries in a signature database. A control flow graph (CFG) is then constructed using basic blocks that represent possible execution paths within each function. These basic blocks are connected by directed edges based on control flow instructions and jump targets, forming the CFG. To capture interactions across functions, interprocedural calls are identified, and edges are added between caller and callee functions, thereby extending the CFG into an ICFG.

For each log operation in *LogOps*, the algorithm updates *Vars* with the variables involved in the log operation (line 6). It then performs a BACKWARDSLICING analysis on the ICFG to extract slices representing the reverse execution flow from the log operation to the function entry point (line 7). These slices are used to derive inter-functional paths (*Paths*) for further analysis (line 8).

For each path p in *Paths*, the algorithm performs taint analysis using the TAINTANALYSIS function (line 9). This function evaluates whether tainted data propagates from the log operation to the path entry point. Starting from the log operation, tainted variables are tracked by propagating taint through instructions in the reverse execution path. Specifically, if an instruction interacts with tainted variables, its associated variables are added to the set of tainted variables. The process continues until reaching the path entry point, which serves as the source of the taint. If tainted variables are found at the path entry, the function returns the entry point as the ($source_{op}$); otherwise, it returns `null`.

If a source of tainted data ($source_{op}$) is identified, the algorithm checks if there are any storage operations (`sstore`) in the portion of the path leading to the source (line 11). Additionally, it verifies whether the variables involved in these `sstore` operations are subject to constraints (e.g., conditional jumps like `jumpi`). If no `sstore` operations are found, or if the `sstore` operations lack constraints on their variables, the corresponding event is added to E_v as potentially vulnerable to *Inconsistent Logging* (line 12). The event is also recorded in a separate set of tainted paths for further evaluation (line 13).

If no source of tainted data is identified, the algorithm checks if the path contains an external call (`call`) without corresponding constraints (e.g., conditional branches like `jumpi`) (line 15). If such conditions are found, the event is added to E_v as vulnerable to *Event Counterfeiting* (line 16).

After analyzing all paths for a log operation, the algorithm checks if multiple tainted paths exist for the same event (line 18). If so, the event is flagged as susceptible to *Event Counterfeiting* due to the presence of multiple emission paths (line 19).

By analyzing the tainted data flow and identifying conditions such as missing constraints, unchecked external calls, and the absence of storage operations, the algorithm effectively detects vulnerabilities. Events flagged in E_v are susceptible to either *Event Counterfeiting*, where the same event may be misleadingly emitted from multiple paths, or *Inconsistent Logging*, where logged data may lack of control.

2) Source-Code-Level Validation of Event Parameter Constraints: Bytecode-level analysis is limited as it only reveals indexed event parameters, without access to the full semantics of events. Consequently, bytecode-level analysis can only detect potential *Inconsistent Logging* issues by identifying inconsistencies in indexed parameters, and it cannot address non-indexed parameters. To achieve complete validation, source code analysis is necessary. At the source code level, we gain access to both indexed and non-indexed parameters, enabling a comprehensive evaluation of potential vulnerabilities, including both *Event Counterfeiting* and *Inconsistent Logging*, which bytecode-level analysis alone cannot fully uncover.

To address these challenges, we rely on the full semantic context of the contract source code to trace event-related call paths and apply symbolic execution to verify parameter constraints. This ensures that events are emitted with consistent values across all paths, aligning both indexed and non-indexed parameters with the intended contract logic. Additionally, *Event Counterfeiting* detection at the source-code level involves analyzing multiple execution paths to verify that events are not improperly used across functions, providing a depth of validation that cannot be achieved with bytecode analysis alone.

At the source-code level, as detailed in Alg. 2, the tool begins by initializing an empty set E_v to store potentially vulnerable events. Then extracts all events E from the smart contract source code *SC*. For each event in E, the tool performs the following steps:

First, the tool extracts all execution paths $Paths$ in *SC* that lead to the emission of the event using the SEARCHPATHS function. This involves traversing the contract's call graph to capture all inter-procedural paths from user-callable functions to the event-emitting statements. For each path $p \in Paths$, the tool performs symbolic execution using SYMBOLICEXEC to extract the path constraints related to the event parameters. The tool then applies the CHECKLOGGINGINCONSISTENCY function to verify if any path shows inconsistencies in logging

Algorithm 2 Finding Inconsistent Logging and Event Counterfeiting via Symbolic Execution

Input: SC, the source code of smart contracts.
Output: E_v, a set of phantom events.
$E_v = \emptyset$
extract E, i.e., all events from SC.
for $event \in E$ **do**
    $Paths \leftarrow$ SEARCHPATHS$(event, SC)$ ▷ Get all paths reaching the event
    for $p \in Paths$ **do**
        $constraints \leftarrow$ SYMBOLICEXEC(p)
        if CHECKLOGGINGINCONSISTENCY(constraints) **then**
            $E_v \leftarrow E_v \cup \{event\}$
    for $p_1, p_2 \in Paths$ **do**
        $constraints_1 \leftarrow$ SYMBOLICEXEC(p_1)
        $constraints_2 \leftarrow$ SYMBOLICEXEC(p_2)
        if SMT-SOLVE$(constraints_1 \wedge constraints_2)$ **then**
            $E_v \leftarrow E_v \cup \{event\}$
return E_v

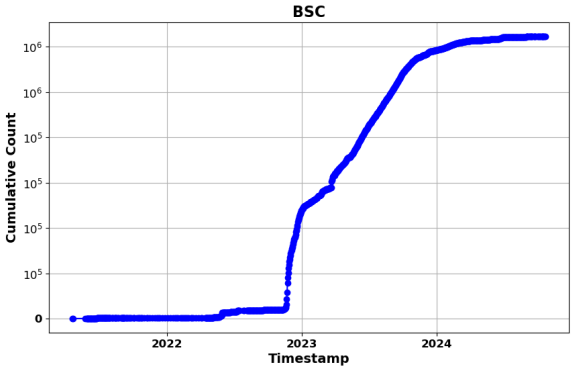
by comparing constraints for all parameters (both indexed and non-indexed). This function is similar to the bytecode analysis, it will detect vulnerabilities by checking for missing constraints on variables, unchecked external calls, or missing storage operations. If inconsistencies are detected, the event is added to E_v as potentially vulnerable to *Inconsistent Logging*.

Next, the tool performs *Event Counterfeiting* detection by considering all pairs of distinct paths (p_1, p_2) in $Paths$ which have the same event. For each pair, it performs symbolic execution to obtain constraints $constraints_1$ and $constraints_2$ for each path. Using an SMT solver, it checks if the conjunction $constraints_1 \wedge constraints_2$ is satisfiable. If the solver finds a solution, this indicates that there is an overlap in parameter values between the two paths, making it difficult for validators to distinguish between events emitted from different paths. In such cases, the event is added to E_v as potentially susceptible to *Event Counterfeiting*.

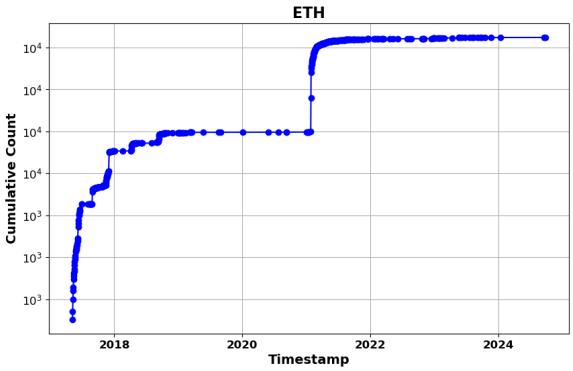
Finally, the tool returns the set E_v, which contains events flagged for potential vulnerabilities. For example, consider the *Deposit* event from Fig. 4, which is emitted by the functions *deposit* and *depositETH*. The tool identifies these functions in the AST of the contract, then analyzes their respective execution paths to extract the constraints for the event parameters. For *depositETH*, the constraint is $msg.value > 0$, while for *deposit*, the constraint is bool(token.call(signature)) $\wedge$ $(amount > 0)$. Using an SMT solver, the tool evaluates these constraints to check for any overlapping parameter values. If an intersection exists, this suggests potential *Event Counterfeiting*, as a validator may be unable to distinguish between emissions from these two functions, leading the tool to flag the event as forgeable for further review.

VI. Implementation and Experiments

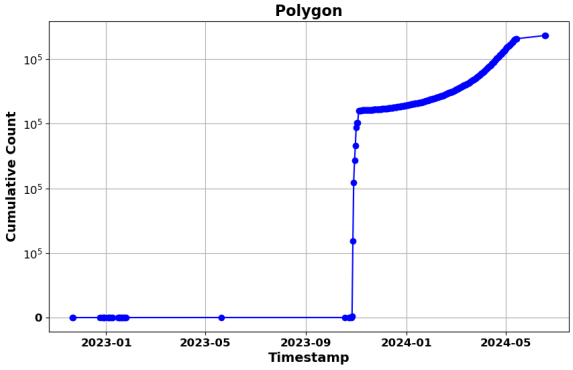
Our tool is implemented in Python and supports both bytecode-level and source code-level analysis for smart contracts, as well as transaction-level monitoring. For bytecode-level analysis, we use the Gigahorse framework to construct the ICFG from compiled bytecode. At the source code level, our tool supports all versions of Solidity compatible with Slither [33], allowing us to extract the ASTs and perform



(a) BSC



(b) ETH



(c) Polygon

Fig. 7: Analysis of ERC-20 transfer phantom events across different platforms for 9 addresses.

detailed analysis on event emissions and parameter constraints. We employ the Z3 [7] as our constraint solver to handle symbolic execution and eliminate infeasible paths. Additionally, the tool includes a transaction-level off-chain monitoring system that parses on-chain data and applies custom rules to detect anomalies and verify the success of attack transactions. All major components, including the bytecode analysis engine, source code symbolic executor, and transaction monitoring system, were developed by us. The implementation is based in Python and includes over 5,000 lines of code.

- **RQ1** *How prevalent are phantom events on major*

blockchain platforms, and how does PEVENTCATCHER *perform in transaction-level detection?* We aim to understand the need for validators to verify the origins of events through contract addresses and source functions.
- **RQ2** *How effective is* PEVENTCATCHER *in detecting potential phantom events from smart contract?* We developed PEVENTCATCHER and constructed a custom benchmark to evaluate its effectiveness.
- **RQ3** *How feasible is it to perform such attacks in real world?* We aim to validate whether the identified vulnerabilities can be performed in real-world scenarios.

A. Prevalence of Phantom Events (RQ1)

Tools like TokenScope [59] have contributed to detecting inconsistencies in token behavior, and Guoyi Ye et al. [51] have demonstrated the prevalence of forged user addresses for transfer events. We aim to explore two specific aspects: the prevalence of suspicious addresses in phantom event attacks, and whether such attacks are common in cross-chain bridges.

1) Methodology: To better understand the real-world occurrence of phantom events, we conduct two separate experiments using blockchain transaction data.

In the first experiment, we focused on nine particular forged addresses, specifically from *0x1111...1111* through *0x9999...9999*, as these addresses are highly unlikely to be generated or used in normal transactions. We examined the transactions up to October 21, 2024, on the Ethereum, BSC, and Polygon blockchains, looking for ERC-20 token movements originating from these addresses. This approach helps measure the prevalence of phantom events across different blockchain platforms.

In the second experiment, we created detection rules for cross-chain bridges based on our transaction-level analysis, given that bridges are highly vulnerable to phantom event attacks. We compared our findings with those of XScope [24], the only existing tool for detecting bridge attack transactions, using its associated dataset.

2) Result: In our first experiment, focusing on nine specific addresses, we identified a significant occurrence of phantom events in ERC-20 token transfers, as shown in Fig. 7. We found 18,099 such events on Ethereum, 1,245,125 on BSC, and 436,407 on Polygon. Such events, if not properly verified, could lead to security exploits. This significant presence from just nine addresses highlights the critical need for rigorous transaction verification to maintain the security and trustworthiness of blockchain operations.

In the second experiment (the result shown in Table III), we detected all transactions reported by XScope and identified additional attack transactions. XScope only records transactions where attacks have already taken place, indicated by a fund transfer on the destination chain. In contrast, we were able to detect attacks before a transfer occurred, helping project teams identify attackers proactively. We also detected attack transactions on meter.io [45], which XScope mentioned in its attack incident list but did not analyze. Additionally, we discovered an attack transaction on CENNZnet, which resulted in a loss of 150 ETH. This incident had not been previously disclosed or reported by any other tools or security companies.

TABLE III: Number of detected attack transactions and successful attacks by tools.

Project	XScope	PEVENTCATCHER	
	Attacks	Attack Trans.	Successful Attacks
THORChain #1	9	9	9
THORChain #2	41	48	41
pNetwork	3	5	3
Qubit Bridge	20	20	20
meter.io	N/A	5	5
CENNZnet	N/A	1	1

Answer to RQ1: *Our analysis reveals a significant presence of phantom events across Ethereum, BSC, and Polygon, based on just nine addresses. Furthermore, our detection rules demonstrated effectiveness compared to state-of-the-art tools.*

B. Phantom Event Detection from Code (RQ2)

1) Datasets: SmartAxe [53] and XGuard [54] are currently the two known tools for detecting cross-chain vulnerabilities, with SmartAxe focusing on bytecode-level analysis and XGuard providing source code-level analysis.

In our analysis of the SmartAxe dataset, we identified that only 8 of the 20 attacks were caused by *Event Counterfeiting*, with vulnerable function path pairs existing within bridge contracts. We re-labeled these 8 attacks and added 29 more function path pairs affected by *Event Counterfeiting* from the 129 bridges analyzed by SmartAxe. To complement this, we selected 500 BSC smart contracts with low similarity (using the TF-IDF algorithm) from the 452,666 verified contracts in BSC [38] and manually annotated them. After deduplication, we identified 80 additional function pairs affected by *Event Counterfeiting*.

By combining SmartAxe and XGuard datasets, along with manually annotated function pairs, we assembled a comprehensive data set containing 117 function pairs affected by *Event Counterfeiting* for evaluating our detection tool.

To detect *Inconsistent Logging*, we used a dataset of 28 manually audited cases of *Inconsistent Logging* parameters, which were identified during our audits.

2) Result: In our experiments, PEVENTCATCHER significantly outperformed SmartAxe and XGuard in detecting both *Event Counterfeiting* and *Inconsistent Logging* vulnerabilities, as shown in Table IV. For *Event Counterfeiting*, SmartAxe achieved an F1 score of 31.01% with a precision of 40.50% and a recall of 25.12%, while XGuard reached an F1 score of 67.70% with a precision of 87.75% and a recall of 55.12%. In comparison, PEVENTCATCHER demonstrated superior performance, achieving an F1 score of 91.30% at the bytecode level and 88.62% at the source code level. For *Inconsistent Logging* detection, PEVENTCATCHER also outperformed XGuard, achieving an F1 score of 94.9% with perfect recall, compared to XGuard's 75.82%.

The difference in performance metrics for PEVENTCATCHER 's bytecode and source code implementations highlights specific limitations in each approach. In *Event Counterfeiting* detection, PEVENTCATCHER 's bytecode-level analysis lacks constraint

TABLE IV: Tools Comparison for Event Counterfeiting and Inconsistent Logging

Tool	Event Counterfeiting			Inconsistent Logging		
	Precision	Recall	F1	Precision	Recall	F1
SmartAxe	40.50%	25.12%	31.01%	N/A	N/A	N/A
XGuard	87.75%	55.12%	67.70%	64.71%	91.75%	75.82%
PEventCatcher (Bytecode)	84.0%	100%	91.30%	62.50%	16.12%	39.31%
PEventCatcher (Source)	90.83%	86.51%	88.62%	90.30%	100%	94.90%

validation, leading to a lower precision as it may flag events that lack sufficient evidence of forgery. However, at the source code level, PEventCatcher performs comprehensive constraint checks, improving precision significantly. For *Inconsistent Logging*, PEventCatcher 's bytecode analysis is limited to indexed parameters only, which reduces recall as it cannot identify issues related to non-indexed parameters. With source code access, PEventCatcher can analyze both indexed and non-indexed parameters, thus achieving higher recall. In contrast, SmartAxe and XGuard have inherent design limitations; SmartAxe only detects whether two functions emit the same event without considering constraint validation within call paths, while XGuard focuses on constraints within single functions and lacks inter-function analysis. By analyzing both function interactions and call paths, PEventCatcher detects vulnerabilities that the other tools miss, achieving superior results in both *Event Counterfeiting* and *Inconsistent Logging* detection.

3) Limitations: The false positives and false negatives in our evaluation revealed two main limitations of our approach. First, when dealing with incomplete contract code, such as interface calls in certain functions, our computed constraints may not always be precise, leading to weaker constraints and thus false positives. Second, compilation errors occurred with some smart contracts that were verified by blockchain explorers but failed to compile with our tool for analysis [39], due to Solidity compile bugs. Addressing these limitations will be a focus of our future work.

Answer to RQ2: PEventCatcher *has proven highly effective in detecting phantom event vulnerabilities, surpassing current state-of-the-art tools in precision, recall, and F1 score.*

C. Real-World Attack Feasibility (RQ3)

1) Methodology: We applied our tool to conduct testing on various blockchain platforms, aiming to identify vulnerabilities related to phantom event attacks in smart contracts. Beyond on-chain testing, we also audited various off-chain applications, including blockchain explorers, NFT marketplaces, and cryptocurrency wallets. These audits focused on evaluating security protocols and identifying potential vulnerabilities that could be exploited by phantom events, ensuring a comprehensive analysis of both on-chain and off-chain components.

All identified phantom event vulnerabilities were tested in a controlled local environment using simulated scenarios to ensure no disruption or harm to real-world blockchain platforms.

2) Result: In our experiments, we identified a *Transfer Event Spoofing* transaction in which a fake transfer appeared to move tokens from the address *0x8888...8888* to the victim's wallet. Despite no actual token transfer occurring, at least five major blockchain explorers—BscScan, OKLink, Bitquery, Tokenview, and Bsctrace [1], [2], [11], [12], [16]—incorrectly recognized this spoofed event as a legitimate transaction. This highlights the widespread vulnerability in off-chain systems, which struggle to differentiate between genuine and phantom events.

Additionally, we successfully replicated the "sleepminting" attack on testnet environments for two leading NFT marketplaces, Opensea and Rarible. Despite the existence of monitoring solutions designed to detect such attacks, fully mitigating these vulnerabilities remains an ongoing challenge.

Moreover, during our security audits, we discovered critical vulnerabilities in six cryptocurrency wallets, which we reported to the respective project teams or through bug bounty platforms. Four of these vulnerabilities were confirmed, with one resulting in a $600 bounty from the project team.

In our audit of several blockchain bridges, we identified vulnerabilities in off-chain code that left them exposed to the Mimicry Contract Attack. In this attack, malicious contracts emit forged events that are incorrectly processed as legitimate by off-chain systems. One vulnerable system we uncovered was forked and deployed by a public blockchain project with a market capitalization exceeding $250 million as of October 9, 2024. These vulnerabilities have been reported to the respective project teams for remediation.

Additionally, we identified a display issue with event data in the popular blockchain explorer Blockscout [60], which could compromise the accuracy of transaction records. This flaw could potentially lead to user misinterpretation of blockchain data, undermining trust in transaction history.

In our audit of multiple active smart contract projects, we also found that three DeFi projects and one GameFi project were vulnerable to the *Inconsistent Logging* attack, a vulnerability first proposed in this paper (see IV-A2). The highest affected market capitalization in this set of projects was $169,688.

Answer to RQ3: *Our research demonstrates that phantom event attacks are feasible on real-world blockchain platforms, specifically targeting blockchain explorers, NFT marketplaces, DeFi, GameFi, and cryptocurrency wallets. This emphasizes the urgent need for improved security measures within the blockchain ecosystem. Detailed reports and confirmation cases are available in the*

supplementary materials.

D. Threats to Validity

The internal validity threats primarily stem from potential inaccuracies introduced during the manual data labeling process. To improve the accuracy of data labeling, we adopted a three-tier approach, where two authors independently labeled the data and a third author reviewed their labels. This method involved a thorough review at three distinct levels to enhance the precision of our dataset categorization.

To ensure external validity, we diversified the types and sources of datasets used in our experiments. In addition to the bridge-focused data provided by SmartAxe and XGuard, we expanded our dataset to include a broader range of scenarios. Furthermore, we avoided using smart contracts with high similarity to improve the diversity and validity of our analysis.

VII. Related Work

A. Smart Contract Security Analysis

In recent years, a large number of techniques have been proposed to analyze the security of smart contracts.

Static Analysis. Tikhomirov et al. [34] designed SmartCheck, a system translating Solidity source code to XML and detecting bugs via xPath patterns. Grech et al. [32] proposed a static analysis framework termed Gigahorse that translates the stack-based bytecode to the register-based intermediate representation. A similar tool named Slither [33] was proposed by Fesit that targets solidity source code. Lu et al. [40] design a smart contracts security analysis tool named NeuCheck which is based on syntax tree parsing. Ghaleb et al. [63] proposed eTainter, a static analysis tool that detects gas-related vulnerabilities in smart contracts by applying taint tracking to bytecode.

Symbolic Execution. Luu et al. [19] proposed Oyente, a pioneering symbolic execution tool for Ethereum smart contracts to detect bugs. Lin et al. [4] presented SolSEE, the first source-level symbolic execution engine for Solidity smart contracts. Ma et al. [35] introduced Pluto for detecting security bugs by reconstructing inter-contract CFGs. Pasqua et al. [26] proposed a method based on the symbolic execution of EVM operands for precise CFG construction and improved vulnerability detection. Ruaro et al. [61] implement CRUSH, which leverages symbolic execution and program slicing to detect storage collisions among such contract groups. Gritti et al. [62] developed JACKAL, which performs symbolic execution based on control flow graphs (CFG) and function call graphs (FCG) to detect confused contract vulnerability. In addition, industry solutions such as Mytrhil [27] and Manticore [36] have become standard tools for smart contract audits.

Our approach involves developing a detection framework that combines multiple techniques, including bytecode-level analysis, source code analysis, transaction-level monitoring, and symbolic execution, but the issues addressed in this paper cannot be captured by any existing vulnerability patterns.

B. Smart Contract Events

Generally, logging messages enhances program comprehension and reduces maintenance costs, but research on Solidity event logging and security is still limited. Li et al. [23] conducted the first empirical study on Solidity event logging and developing a tool to identify the event that causes unnecessary gas usage. Zhang et al. [24] designed a tool named Xscope which finds security violation events in cross-chain bridges. Cernera et al. [25] tracked the tokens created by internal transactions by scanning the logs looking for Transfer events. Additionally, Guidi et al. [46] analyzed the phenomenon of sleepminting and explored the use of Forta for tracking and alerting about suspicious events. Zhu et al. [5] proposed a technique called DocCon, which detects inconsistencies between Solidity code and its corresponding documentations. These inconsistencies include documented event emissions which do not appear in code, or vice versa. TokenScope [59] contributed to the detection of inconsistencies in token behaviors by monitoring ERC-20 events. Despite these advancements, most research has only considered issues related to events in specific scenarios and has not fully addressed the generality of attacks caused by phantom events.

VIII. Mitigation Strategies

Mitigating vulnerabilities caused by phantom events requires a comprehensive approach, addressing smart contract development, ecosystem infrastructure, and attack detection mechanisms. From the perspective of contract development, developers should implement strict validation mechanisms to ensure that event parameters are verified before emission and access control mechanisms for the functions. It is essential to enforce proper state transitions to prevent mismatches between emitted events and the actual contract state.

At the ecosystem level, off-chain systems like blockchain explorers, wallets, and DApps must adopt more robust validation techniques to distinguish legitimate events from phantom events. Event emitter validation, where the source of the event is cross-checked with the contract address, helps ensure that events originate from authorized contracts. Furthermore, improving data sanitization processes in off-chain applications is critical to prevent vulnerabilities such as cross-site scripting (XSS) and SQL injection (SQLi). Enhanced cross-chain security protocols are necessary for cross-chain bridges, ensuring that events on both the source and destination chains are validated to prevent event forgery and manipulation.

In terms of security attack detection, continuous real-time monitoring of on-chain transactions and events is essential to detect and flag suspicious activities, such as *Transfer Event Spoofing* or *Contract Imitation*. Defining detailed detection rules, both for on-chain contract behavior and off-chain event handling, allows for more comprehensive identification of vulnerabilities. Additionally, regular security audits of both smart contracts and off-chain systems should be conducted to identify potential weaknesses, particularly focusing on event emission logic, access controls, and transaction validation. Through a combination of these strategies, the risk posed by phantom events can be significantly reduced, improving the security and reliability of blockchain systems.

IX. Conclusion

In this work, we conducted an in-depth analysis of vulnerabilities associated with phantom events in blockchain systems. Our approach involved developing a multi-level detection framework that integrates bytecode-level, source code-level, and transaction-level analyses to address issues such as *Event Counterfeiting*, *Inconsistent Logging*, and *Contract Imitation*. Furthermore, we identified real-world cases for each of the proposed attack vectors, underscoring the critical need for mitigation of these threats in blockchain ecosystems.

References

[1] Bitquery, "Bitquery", 2023. [Online]. Available: https://explorer.bitquery.io/. [Accessed: Dec. 15, 2023].

[2] Tokenview, "Tokenview", 2023. [Online]. Available: https://bsc.tokenview.io/. [Accessed: Dec. 15, 2023].

[3] Chainlink, "Chainlink", 2023. [Online]. Available: https://chain.link/. [Accessed: Dec. 15, 2023].

[4] S.-W. Lin, P. Tolmach, Y. Liu, and Y. Li, "SolSEE: a source-level symbolic execution engine for Solidity", in *Proc. 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 1687–1691.

[5] C. Zhu, Y. Liu, X. Wu, and Y. Li, "Identifying Solidity Smart Contract API Documentation Errors", in *Proc. 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.

[6] H. Kamarul, "'Spoof' Tokens on Ethereum", 2022. [Online]. Available: https://medium.com/etherscan-blog/spoof-tokens-on-ethereum-c2ad882d9cf6. [Accessed: Dec. 15, 2023].

[7] L. De Moura and N. Bjørner, "Z3: An efficient SMT solver", in *Proc. International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, 2008, pp. 337–340.

[8] Halborn, "Thorchain Incident Analysis", 2023. [Online]. Available: https://github.com/HalbornSecurity/PublicReports/blob/master/IncidentReports/. [Accessed: Dec. 15, 2023].

[9] PeckShield, "PeckShieldAlert on X", 2023. [Online]. Available: https://twitter.com/PeckShieldAlert/status/1551630862600527874?t=pAmAETpV2jpmLAgU23QD9w&s=05. [Accessed: Dec. 15, 2023].

[10] B. Guidi and A. Michienzi, "Delving NFT vulnerabilities, a sleepminting prevention system", *Multimedia Tools and Applications*, pp. 1–20, 2023.

[11] Oklink, "OKLINK", 2023. [Online]. Available: https://www.oklink.com/. [Accessed: Dec. 15, 2023].

[12] Bsctrace, "Bsctrace", 2023. [Online]. Available: https://bsctrace.com/. [Accessed: Dec. 15, 2023].

[13] I. Ilascu, "OpenSea NFT platform bugs let hackers steal crypto wallets", 2022. [Online]. Available: https://www.bleepingcomputer.com/news/security/opensea-nft-platform-bugs-let-hackers-steal-crypto-wallets/. [Accessed: Dec. 15, 2023].

[14] B. Mueller, "Rektosaurus", 2023. [Online]. Available: https://github.com/muellerberndt/rektosaurus. [Accessed: Dec. 15, 2023].

[15] I. Visconti, A. Vitaletti, and M. Zecchini, "Preventing Content Cloning in NFT Collections", in *Proc. International Conference on Applied Cryptography and Network Security*, 2023, pp. 84–99.

[16] Bscscan, "BNB Smart Chain (BNB) Blockchain Explorer", 2023. [Online]. Available: https://bscscan.com/. [Accessed: Dec. 15, 2023].

[17] Knownsec 404 Blockchain Security Research Team, "Blockwell.ai KYC Casper Token 'Psoriasis Advertising' Event Analysis", 2023. [Online]. Available: https://paper.seebug.org/709/. [Accessed: Dec. 15, 2023].

[18] GiorgioHerbie, "Macchina virtuale Ethereum (EVM)", 2022. [Online]. Available: https://ethereum.org/it/developers/docs/evm/. [Accessed: Dec. 15, 2023].

[19] B. Bünz, L. Kiffer, L. Luu, and M. Zamani, "Flyclient: Super-light clients for cryptocurrencies", in *Proc. IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 928–946.

[20] A. M. Antonopoulos and G. Wood, *Mastering Ethereum: Building Smart Contracts and Dapps*, O'Reilly Media, 2018.

[21] N. Ivanov, J. Lou, T. Chen, J. Li, and Q. Yan, "Targeting the weakest link: Social engineering attacks in Ethereum smart contracts", in *Proc. ACM Asia Conference on Computer and Communications Security*, 2021, pp. 787–801.

[22] Polygonscan, "ethAddress.io (ethAddress.io) Token Tracker — PolygonScan", 2022. [Online]. Available: https://mumbai.polygonscan.com/token/0x325f188ce09f94bd8ea7bf395019cd146780ba20?a=0x888888888888888888888888888888888888. [Accessed: Dec. 15, 2023].

[23] L. Li, Y. Liang, Z. Liu, and Z. Yu, "Understanding Solidity Event Logging Practices in the Wild", in *Proc. 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 300–312.

[24] J. Zhang, J. Gao, Y. Li, Z. Chen, Z. Guan, and Z. Chen, "Xscope: Hunting for cross-chain bridge attacks", in *Proc. 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–4.

[25] F. Cernera, M. La Morgia, A. Mei, and F. Sassi, "Token Spammers, Rug Pulls, and Sniper Bots: An Analysis of the Ecosystem of Tokens in Ethereum and in the Binance Smart Chain (BNB)", in *Proc. 32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 3349–3366.

[26] M. Pasqua, A. Benini, F. Contro, M. Crosara, M. Dalla Preda, and M. Ceccato, "Enhancing Ethereum smart-contracts static analysis by computing a precise Control-Flow Graph of Ethereum bytecode", *Journal of Systems and Software*, vol. 200, pp. 111653, 2023.

[27] Consensys, "Mythril", 2023. [Online]. Available: https://github.com/ConsenSys/mythril/. [Accessed: Dec. 15, 2023].

[28] R. Behnke, "EXPLAINED: THE PNETWORK HACK (SEPTEMBER 2021)", 2023. [Online]. Available: https://www.halborn.com/blog/post/explained-the-pnetwork-hack-september-2021. [Accessed: Dec. 15, 2023].

[29] N. Semczuk, "What is a rug pull?", 2023. [Online]. Available: https://www.bankrate.com/investing/what-is-a-rug-pull/. [Accessed: Dec. 15, 2023].

[30] C. Dannen, *Introducing Ethereum and Solidity*, vol. 1, Springer, 2017.

[31] Vyperlang, "Vyper", 2023. [Online]. Available: https://github.com/vyperlang/vyper. [Accessed: Dec. 15, 2023].

[32] N. Grech, S. Lagouvardos, I. Tsatiris, and Y. Smaragdakis, "Elipmoc: Advanced decompilation of Ethereum smart contracts", *Proceedings of the ACM on Programming Languages*, vol. 6, no. OOPSLA1, pp. 1–27, 2022.

[33] J. Feist, G. Grieco, and A. Groce, "Slither: a static analysis framework for smart contracts", in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15.

[34] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, "SmartCheck: Static analysis of Ethereum smart contracts", in *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain*, 2018, pp. 9–16.

[35] F. Ma, Z. Xu, M. Ren, Z. Yin, Y. Chen, L. Qiao, B. Gu, H. Li, Y. Jiang, and J. Sun, "Pluto: Exposing vulnerabilities in inter-contract scenarios", *IEEE Transactions on Software Engineering*, vol. 48, no. 11, pp. 4380–4396, 2021.

[36] M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, and A. Dinaburg, "Manticore: A user-friendly symbolic execution framework for binaries and smart contracts", in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 1186–1189.

[37] Coinware, "Beware of the LayerZero Token Scam: Stay Informed and Stay Safe", 2022. [Online]. Available: https://coinwire.com/beware-of-the-layerzero-scam-stay-informed-and-safe/. [Accessed: Dec. 15, 2023].

[38] Tangtj, "BSC Contract Database", 2023. [Online]. Available: https://github.com/tangtj/bsc-contract-database. [Accessed: Dec. 15, 2023].

[39] SvenMeyer, "Compiling 16 Files with 0.7.6 CompilerError", 2023. [Online]. Available: https://github.com/ethereum/solidity/issues/11638. [Accessed: Dec. 15, 2023].

[40] N. Lu, B. Wang, Y. Zhang, W. Shi, and C. Esposito, "NeuCheck: A more practical Ethereum smart contract security analysis tool", *Software: Practice and Experience*, vol. 51, no. 10, pp. 2065–2084, 2021.

[41] Forta, "Forta Network", 2023. [Online]. Available: https://forta.org/. [Accessed: Dec. 15, 2023].

[42] V. Buterin et al., "A next-generation smart contract and decentralized application platform", *White Paper*, vol. 3, no. 37, pp. 2–1, 2014.

[43] X. Tang, Z. Jia, and W. Yang, "Blockchain Application Status and Ecology", in *Blockchain Application Guide: Methodology and Practice*, pp. 35–48, Springer, 2022.

[44] R. Behnke, "EXPLAINED: THE QUBIT HACK (JANUARY 2022)", 2022. [Online]. Available: https://www.halborn.com/blog/post/explained-the-qubit-hack-january-2022. [Accessed: Dec. 15, 2023].

[45] R. Behnke, "EXPLAINED: THE METER.IO HACK (FEBRUARY 2022)", 2022. [Online]. Available: https://www.halborn.com/blog/post/explained-the-meter-io-hack-february-2022. [Accessed: Dec. 15, 2023].

[46] B. Guidi and A. Michienzi, "Sleepminting, the brand new frontier of Non Fungible Tokens fraud", in *Proceedings of the 2022 ACM Conference on Information Technology for Social Good*, 2022, pp. 75–81.

[47] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system", 2008.
[48] CoinMarketCap, "Bitcoin price today, BTC to USD live price, marketcap and chart — CoinMarketCap", 2024. [Online]. Available: https://coinmarketcap.com/currencies/bitcoin/. [Accessed: Dec. 15, 2023].
[49] Scam Sniffer, "Zero Transfer Scam". [Online]. Available: https://twitter.com/realScamSniffer/status/1755862295224459675. [Accessed: Oct. 15, 2024].
[50] Dune, "How Users React". [Online]. Available: https://dune.com/kimichi/how-users-react. [Accessed: Oct. 15, 2024].
[51] G. Ye et al., "Interface Illusions: Uncovering the Rise of Visual Scams in Cryptocurrency Wallets", *Proceedings of the ACM on Web Conference 2024*, 2024.
[52] N. Grech, L. Brent, B. Scholz, et al., "Gigahorse: Thorough, declarative decompilation of smart contracts", in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, IEEE, 2019, pp. 1176–1186.
[53] Z. Liao, Y. Nan, H. Liang, et al., "Smartaxe: Detecting cross-chain vulnerabilities in bridge smart contracts via fine-grained static analysis", *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 249–270, 2024.
[54] K. Wang, Y. Li, C. Wang, et al., "XGuard: Detecting Inconsistency Behaviors of Crosschain Bridges", in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 612–616.
[55] BscScan, "TroyEmpire", 2024. [Online]. Available: https://bscscan.com/address/0xb48200ed722e7e86a78d04245bf743d047289e95#code. [Accessed: Jun. 7, 2024].
[56] BscScan, "SecondLive", 2024. [Online]. Available: https://bscscan.com/address/0x260e69ab6665b9ef67b60674e265b5d21c88cb45#code. [Accessed: Jun. 7, 2024].
[57] BscScan, "TTGame", 2024. [Online]. Available: https://bscscan.com/address/0x7192611537108231ce07ac20ddf40c850a00ef6a#code. [Accessed: Jun. 7, 2024].
[58] BscScan, "WalletLocking", 2024. [Online]. Available: https://bscscan.com/address/0xb89cb9297c29ca0b4ae1d2f0b68089c9f39017ce#code. [Accessed: Jun. 7, 2024].
[59] T. Chen, Y. Zhang, Z. Li, et al., "TokenScope: Automatically detecting inconsistent behaviors of cryptocurrency tokens in Ethereum", in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1503–1520.
[60] Our Team, "Error in displaying msg.sender in Transaction Logs", 2024. [Online]. Available: https://github.com/blockscout/blockscout/issues/9879. [Accessed: Jun. 7, 2024].
[61] Ruaro, Nicola and Gritti, Fabio and McLaughlin, et al., "Not your Type! Detecting Storage Collision Vulnerabilities in Ethereum Smart Contracts", in *Netw. Distrib. Syst. Security Symp*, 2024.
[62] Gritti, Fabio and Ruaro, Nicola and McLaughlin, et al., "Confusum contractum: confused deputy vulnerabilities in ethereum smart contracts", in *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 1793–1810, 2023.
[63] Ghaleb, Asem and Rubin, Julia and Pattabiraman, et al., "eTainter: detecting gas-related vulnerabilities in smart contracts", in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 728–739, 2022.

APPENDIX A
REAL-WORLD ATTACK EXAMPLES

A. Examples of the Five Attack Vectors in Our Taxonomy

Event Counterfeiting. Two well-known instances of *Event Counterfeiting* are the attacks on Qubit Bridge [28] and Meter Bridge [45], which resulted in losses of $80 million and $4.3 million, respectively.

Inconsistent Logging. Many of these issues were discovered from the 25 most active smart contracts on the BSC chain on 1 April 2024. Specifically, the contracts for TroyEmpire [55], SecondLive [56], WalletLocking [58], and TTGameEvents [57] were found to have these issues. For example, TTGame project generates an address for users' registration and transfers a certain amount of BNB to the address as gas fees for future automatic calls. By identifying authorized user addresses from the historical transactions, we can find the remaining unauthorized transactions that emit phantom events. Although user permissions for the other three projects were unclear, we can simulate to call their functions and arbitrarily generate withdrawal events, confirming the existence of these issues.

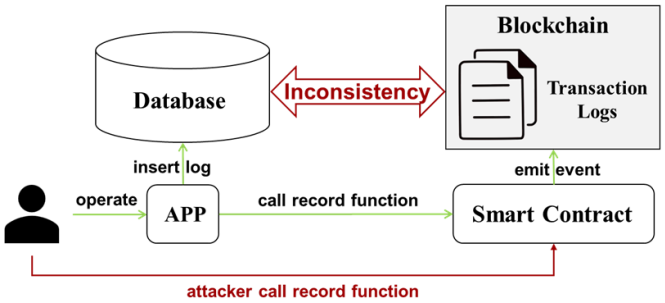


Fig. 8: The overview of Inconsistent logging apps.

Contract Imitation. For *Contract Imitation* attack, the most notable instance is the attack on pNetwork, which resulted in a loss of $12.7 million. This has already been briefly introduced in Sect. III-C. Similarly, THORChain experienced a loss of $8 million due to an error handler issue [8]. As for mimicry contract attack, a representative case is the "sleepminting" attack, which gained attention for generating an NFT valued at $69 million and listing it on a platform. This incident has been widely studied in the academic community [10], [15], [46]. According to recent information from the Forta platform [41], there were 8,130 sleepminting alerts generated in just 7 days, from December 3 to December 9, 2023.

Transfer Event Spoofing. Our analysis of on-chain data revealed numerous instances of *Transfer Event Spoofing*, where ERC-20 token and NFT events had altered sender addresses. These included forged addresses resembling celebrities, well-known exchanges, and other eye-catching entities. Several of these cases have led to significant financial losses [9], [22], [37], [49], [51].

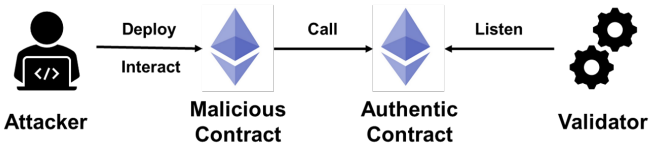
Event Handling Error. A well-known example of *Event Handling Error* involves the misinformation case experienced by Rarible and OpenSea, caused by sleepminting. Blockchain explorers like Etherscan and Bscscan, as well as various wallets, often treat forged transfer events as legitimate transactions. Another instance involves blockwell.ai, where wallets mistakenly identified a phantom event from a spoofed token as a valid ERC-20 transfer [17]. Similar vulnerabilities have been observed in the ERC-721 and ERC-1155 tokens.

In addition, OpenSea, the largest NFT marketplace, has faced similar issues. The attackers manipulated the metadata of the NFTs displayed by OpenSea, embedding malicious payloads that triggered unwanted wallet behaviors, ultimately allowing attackers to profit. Similar attacks have been found in ERC-20 projects, highlighting the widespread difficulty in securely handling event data across the blockchain ecosystem [13], [14].

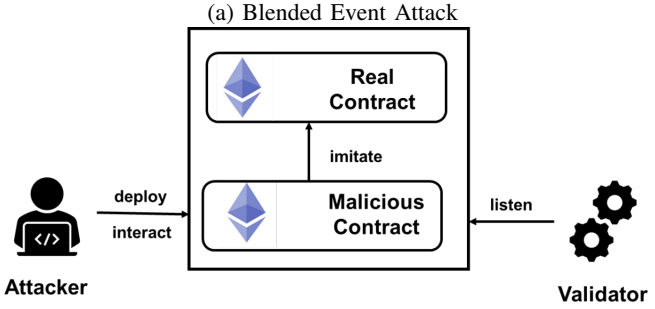
B. Framework of Inconsistent Logging and Contract Imitation

Figure 8 provides an overview of *Inconsistent Logging* in applications. In this framework, a user operates through

an application, which interacts with a database to insert logs and with a smart contract on the blockchain to emit events. However, inconsistencies can arise between the database logs and the blockchain transaction logs due to discrepancies in logging practices. Attackers can exploit this by directly calling the record function, potentially creating mismatches between the application's internal log records and the blockchain's transaction logs.



(a) Blended Event Attack



(b) Mimicry Contract Attack

Fig. 9: Two type of Contract imitation

Figure 9 illustrates two types of contract imitation attacks. In the *Blended Event Attack*, a malicious contract interacts with an authentic contract, and logs from both contracts are recorded in the same transaction, which may mislead validators. In the *Mimicry Contract Attack*, the attacker deploys a contract that imitates a real contract, generating deceptive logs that appear to originate from the authentic contract, creating further confusion for validators.

C. Case of Transfer Event Spoofing

Figure 10 illustrates an example of a *Transfer Event Spoofing* attack, where the attacker forges the event sender, causing blockchain explorers and wallets to incorrectly display it as a successful transfer. This misrepresentation can deceive users and is used as a social engineering tactic to exploit trust in these interfaces.

The *Transfer Spoofing* attack on 9 February 2024, where an attacker successfully executed a *Transfer Event Spoofing* attack, resulting in a loss of $1.04 million USD. This attack exploited

Fig. 10: Airdrop event spoofing.

TABLE V: Real case of transfer event spoofing attack transactions.

	Sender	Transfer From	Transfer To	Token
1	Victim	Victim	0x734659...50Ca79F7	USDT
2	Attacker	Victim	0x73435A..2Bca79F7	FAKE USDT
3	Victim	Victim	0x73435A...2Bca79F7	USDT

vulnerabilities in wallets and blockchain explorers that parse and display transfer events. As shown in Table V, after a user sent funds to a legitimate address (0x734...a79F7), the attacker forged a transfer event, making it appear as though the funds were sent to a similar address controlled by the attacker. Due to the way certain wallets and explorers processed these events, the fake transfer was displayed as legitimate, leading the user to mistakenly send additional funds to the attacker's address, resulting in significant financial losses [49].